EventOps Intelligence

Engineering Design and Implementation

Technical assessment report | 10 October 2026

1. Scope and delivery status

EventOps Intelligence implements a unified event operations application with structured RAG and an attendee-verification agent. It is designed to demonstrate conventional application engineering alongside semantic retrieval, model tool selection, structured extraction and deterministic business verification.

The local application is operational against PostgreSQL and explicit fixture mode. Account-dependent live integration and deployment requirements remain unverified. This report describes implemented behavior and established local evidence separately from the steps that still need Google, Gemini or hosting authorization.

2. Product journey and frontend

A minimal backend-validated sign-in leads to Overview. The dashboard shows indexed source counts, persisted agent activity, review-required runs and truthful configuration status. There are no fabricated activity entries.

Knowledge Assistant accepts a question and optional saved conversation. It renders the stored answer, support state, original evidence records and vector scores. Conversation selection and browser refresh load PostgreSQL history rather than relying on React component state alone.

Agent Workspace accepts an operational objective and exact event identifier. Run initiation commits an idempotently identified job and returns promptly. The frontend polls status and displays planning, selected tools, execution outcomes, extraction and verification progression. Staff can inspect stage payloads and save an audit note on review-required runs.

Verification Results groups normalized attendee information, explanations and source provenance. Filters separate verified, unverified and review-required records. The CSV endpoint selects verified rows only and neutralizes formula-like strings. Execution History links actual persisted runs back to their detailed evidence.

React Router handles the six views. Fetch includes credentials and mutation headers; unnecessary Content-Type headers were removed from GET requests during browser validation. The custom CSS supports desktop cards, mobile navigation and scrollable result tables, with loading, error and empty states throughout.

3. Domain data and source schemas

The synthetic corpus contains three events, four operational policies and six participants. An additional organization catalog supports the fictional organization model. Calendar-style invitations and registration correspondence provide controlled integration fixtures.

Event records include exact IDs, schedule, timezone, venue, capacity and registration deadline. Policy records include record ID, scope, rule and authority. Directory entries contain record ID, participant ID, name, email, organization, active status and eligible event identifiers.

Pydantic rejects unknown fields and invalid email shapes. Whole records remain traceable by document ID, record ID and original source path. Each fixture email includes original body text plus fixture-only extraction labels; live extraction never reads these labels.

Controlled edge cases are deliberate: duplicate Ben registrations, Cara/Carla name conflict, inactive Daniel, Elena's organization conflict, Gina's missing email and Hugo's absent directory entry. The expected statuses are documented and asserted in tests.

4. Ingestion design

Ingestion validates all three knowledge documents before changing the index. A checksum combines raw content, execution mode, embedding model and schema version. Unchanged documents are skipped, making ingestion repeatable without duplicate chunks.

Every complete structured record becomes one chunk. This preserves relationships between identity, event and policy fields and avoids splitting small JSON records into fragments that lose their meaning. Source metadata retains the complete parsed record, filename and stable record ID.

Live embedding calls use the official Google Gen AI SDK with RETRIEVAL_DOCUMENT task configuration and 768-dimensional Gemini vectors. The boundary checks response count, dimensions and finite values. Document and chunk updates commit atomically after preparation; an embedding or parsing failure does not replace a valid index with incomplete data.

5. Retrieval and grounded answers

Query interpretation uses a structured model response to resolve intent and contextual references from the last eight saved messages. Explicit EVT identifiers are preserved and override inferred identifiers. Event-specific retrieval includes matching events, applicable policies and eligible directory entries while excluding other event records.

Query embeddings use RETRIEVAL_QUERY. The service ranks candidate record vectors by cosine similarity, selects up to eight records above an initial 0.20 threshold and supplies their stable evidence IDs to generation. PostgreSQL JSON vectors are adequate for this thirteen-record corpus; exact scans avoid introducing an approximate index without a scaling need. pgvector is a documented future option.

Gemini's Answer schema contains text, a supported flag and citation IDs. The backend rejects references outside retrieved evidence and supported answers without citations. Unsupported answers return no citations and explain insufficient information. Both question and answer, along with evidence and retrieval details, are saved durably.

Citation membership validates provenance identity, not full semantic entailment. A live evaluation suite includes direct facts, multiple-record questions, ambiguity, unsupported information, exact identifier lookups and conflicting claims. The evaluator measures retrieval recall, support classification, citation validity and expected answer terms; human grounding review remains explicitly required. No live scores are claimed without a configured Gemini account.

6. Agent reasoning and state ownership

LangGraph defines explicit planning, tool coordination, extraction, verification, review and finish nodes. The tool node conditionally loops over the validated plan, and verification conditionally routes to review or a terminal outcome. This is not a single hardcoded successful API sequence.

In live mode, Gemini returns function calls for read_calendar, read_gmail and read_web. Calls must have allowed names, the immutable requested event ID and a maximum of three unique tool selections. The directory tool is mandatory. Calendar and registration evidence are required by deterministic verification even when a narrower objective leads to a smaller plan.

State includes the objective, event, plan/rationale, pending tool names, fetched sources, failures, attendees, result records and final status. LangGraph checkpoints persist to PostgreSQL. The API's run queue stores admission and lifecycle separately from graph checkpoints, allowing a worker to claim and recover work after process interruptions.

The review transition produces a durable review_required outcome. Staff notes provide human escalation and an audit trail. They do not override unresolved verification rules or resume an interactive graph interrupt. Source corrections are followed by a new run with a fresh idempotency key.

7. External integrations and n8n

The direct Calendar adapter reads a dedicated calendar's events using an event-specific search. It retains event IDs, summary, start/end metadata and attendees. Declined invitees are excluded from attendee candidates. Pagination beyond the bounded result limit becomes an explicit failure.

The Gmail adapter searches event-specific subjects, retrieves full message payloads and decodes plain-text MIME parts. Gemini extraction returns validated attendee fields with original message IDs. Source IDs must belong to fetched messages; extracted field strings must occur in the source text. Missing fields remain null, and malformed or unsupported extraction fails rather than being invented.

The HTTP adapter fetches an operator-configured HTTPS JSON directory. It does not accept URLs supplied in objectives or external content. Redirects are disabled; schema, response-size and record-count limits are enforced.

The optional n8n transport invokes protected webhooks with the same event-scoped contract. Calendar and Gmail workflows use OAuth-authenticated Google API HTTP nodes; the directory workflow uses a fixed HTTP Request node. Responses must contain a data array and complete=true. Workflow exports are inactive and credential-free until the operator assigns credentials and publishes them.

LangGraph retains high-level orchestration and verification ownership. n8n owns selected business-data reads. Local infrastructure and workflow imports were verified; actual credentialed n8n calls remain an account-dependent step.

8. Deterministic verification

Normalization applies Unicode normalization, whitespace cleanup and case-normalized email keys. Records are grouped by email, participant ID where necessary, or source ID for otherwise unidentified records. Original submitted values are retained in evidence.

The authoritative directory establishes participant identity, organization, active state and event eligibility. Registration records establish submitted details, while Calendar establishes invitation evidence. Verification requires an active eligible unique directory match, explicit registration, an invitation and consistent identity/organization fields.

Rules detect missing names/emails/identifiers/organizations, invalid email format, duplicate registrations/invitations, conflicting participant IDs, IDs appearing under multiple emails, event mismatches, inactive contacts and absent/ambiguous directory matches. An unavailable source prevents verified status even when surviving records look consistent.

A directory failure causes the run to fail while preserving available evidence. Other source failures can produce partial records requiring review. Every outcome contains specific explanations and source references. The model cannot overwrite these decisions.

9. Persistence, recovery and reliability

SQLAlchemy/psycopg persists application tables in PostgreSQL; LangGraph creates its separate checkpoint schema. Document ingestion metadata, conversation history, queue records, execution events, verification rows, encrypted credentials and final state survive backend restarts.

Run admission uses a unique owner/idempotency-key constraint and rejects conflicting reuse. At most three active objectives are allowed per operator. The worker uses PostgreSQL row locks, a 90-second lease and a 20-second renewal heartbeat. Expired running jobs are reclaimable; pending graph checkpoints resume rather than discarding gathered state.

External requests have explicit connection/read timeouts, bounded retries and backoff, with retry events recorded. Failure transitions never become fabricated successful completion. Execution events are at-least-once around crash recovery, since read-only tool calls may repeat if a process dies before checkpoint commit.

Schema version 1 is reproducibly initialized through additive create_all and exported PostgreSQL DDL. This is not a substitute for future versioned migrations. The current deployment uses a single Uvicorn process and a worker thread; a separate durable worker and queue monitoring are next steps for unattended operations at larger scale.

10. Security design

Application sessions use opaque random HTTP-only cookies with hashed server-side tokens and expiry. The configured demo password stays server-side and is compared with constant-time digest comparison. Mutations require the exact configured Origin; CORS permits one frontend origin and credentialed requests. Development private-network CORS is allowed only for localhost frontend configuration.

Google authorization is separate from application sign-in. The OAuth flow binds a one-use expiring state to the active session, checks consent callbacks and encrypts saved tokens using a persistent Fernet key. Read-only Calendar and Gmail scopes implement least privilege. n8n separately stores its assigned OAuth and webhook secrets in its credential store.

Emails, objectives and webpages are untrusted data. Allowed tool names, immutable event IDs and fixed source locations constrain model-selected operations. There are no email sending or Calendar write capabilities. Operational histories remain authenticated; only the fictional static reference directory is

intentionally publishable.

Before real organizational use, strengthen gateway limits, identity/roles, data retention, relational constraints, secret rotation and operational monitoring. The shared operator and dedicated fictional accounts are explicit assessment boundaries.

11. Validation evidence

Thirty-two backend tests and nine frontend tests passed before final packaging. Backend coverage includes API/session contracts, ingestion idempotency, schema validation, vector ranking, source traceability, unsupported answers, citation rejection, durable history, deterministic edge cases, tool failures, conditional graph routing and checkpoint recovery across graph recreation.

Frontend tests cover authenticated navigation, session handling, loading/errors, API credentials, saved messages/evidence and verification rendering. The strict TypeScript/Vite production build succeeded, and the patched dependency installation reported no known npm vulnerabilities.

A real local HTTP smoke test used PostgreSQL and fixture mode: sign-in, document ingestion, evidence-bearing query, saved conversation, idempotent workflow, agent worker, seven EVT-101 records, one verified CSV row and review-note persistence. A process restart check confirmed session, conversation, run and results survived. Browser verification exposed and corrected a local CORS/request-header issue, then rendered actual database counts and evidence.

These checks establish local contracts and persistence. They do not establish live semantic quality, Google authorization, public HTTP retrieval or successful hosting. Those checks are documented separately and require account access.

12. Deployment and remaining work

The frontend has Vercel SPA configuration and a publishable fictional reference resource. The backend has a Python 3.12 Dockerfile, Render Free blueprint, configurable origins/cookies, health checks and managed PostgreSQL settings. Compose supplies persistent local PostgreSQL and n8n.

No public URL is claimed. The operator must authorize hosting, supply Gemini access and dedicated Google consent, select durable free PostgreSQL and provision durable n8n where an existing free host allows it. Free web-service sleeping can pause in-process workers; checkpoints preserve state but cannot guarantee uninterrupted unattended execution. Third-party cookie blocking requires target-browser testing or a suitable same-site deployment arrangement.

The final live walkthrough must use real source-grounded responses and credentialed tools, capture actual history and exports, and update the evidence report with verified URLs and limitations. Until then, the implementation is locally verified and deployment-ready, while the live assessment requirements remain outstanding.